Coding Challenge

← Back

QUESTIONS

Q1 ✓
String Character Frequency Counter
10 marks

Q2 ✓
Vowel and Consonant Classifier with Statistics
10 marks

Q3 ✓
String Compression and Run-Length Encoding
10 marks

Q4 ✓
Advanced Text Statistics Analyser
10 marks

4/4 answered

Q1 String Character Frequency Counter — 10 marks

Write a Python program that accepts a string from the user and counts the frequency of each unique character (case-insensitive). Display the results in the format: character → count. Sort the output alphabetically by character.

Constraints:

- Ignore spaces in the count.
- Treat uppercase and lowercase as the same character.
- Sort output alphabetically.

Skills Assessed:

- String iteration, lower(), conditionals
- Dictionary or manual counting
- Sorting using loops or sorted()

Input: Hello World

output:

d → 1
e → 1
h → 1
l → 3
o → 2
r → 1
w → 1

Your Code

```
s = input().lower()

freq = {}

for ch in s:
    if ch != ' ':
        if ch in freq:
            freq[ch] += 1
        else:
            freq[ch] = 1

for ch in sorted(freq):
    print(ch, freq[ch])
```

Input

```
Hello World
```

Output

```
d 1
e 1
h 1
l 3
o 2
```

Submitted

Counter
10 marks

Q2
Vowel and Consonant Classifier with Statistics
10 marks

Q3
String Compression and Run-Length Encoding
10 marks

Q4
Advanced Text Statistics Analyser
10 marks

4/4 answered

frequent vowel, (c) the least frequent consonant, and (d) the ratio of vowels to consonants as a decimal rounded to 2 places.

Constraints:

• Case-insensitive analysis.

• Ignore spaces and digits.

• Handle sentences with no vowels or no consonants gracefully.

Skills Assessed:

• Character classification with conditionals

• Counting and comparison logic

• Arithmetic and rounding

Sample Input :

Python Programming

Expected Output : Vowels: 5 | Consonants: 12 Most frequent vowel: o (2) Least frequent consonant: y (1) Vowel-to-Consonant Ratio: 0.42

Your Code

```
s = input().lower()

vowels = 'aeiou'
vowel_count = 0
consonant_count = 0

vf = {}
cf = {}

for ch in s:
    if ch.isalpha():
        if ch in vowels:
            vowel_count += 1
            if ch in vf:
                vf[ch] += 1
            else:
                vf[ch] = 1
        else:
            consonant_count += 1
            if ch in cf:
                cf[ch] += 1
            else:
                cf[ch] = 1

print('Vowels:', vowel_count)
print('Consonants:', consonant_count)

if vf:
    mv = max(vf, key=vf.get)
    print('Most frequent vowel:', mv, '(', vf[mv], ')', sep='')
else:
    print('Most frequent vowel: None')

if cf:
    lc = min(cf, key=cf.get)
    print('Least frequent consonant:', lc, '(', cf[lc], ')', sep='')
else:
    print('Least frequent consonant: None')

if consonant_count > 0:
    print('Vowel-to-Consonant Ratio:', round(vowel_count / consonant_co
else:
    print('Vowel-to-Consonant Ratio: Undefined')
```

Input

```
Python Programming
```

Output

```
Vowels: 4
Consonants: 13
Most frequent vowel:o(2)
Least frequent consonant:y(1)
Vowel-to-Consonant Ratio: 0.31
```

Submitted

Counter
10 marks

Q2
Vowel and Consonant Classifier with Statistics
10 marks

Q3
String Compression and Run-Length Encoding
10 marks

Q4
Advanced Text Statistics Analyser
10 marks

4/4 answered

replacing consecutive repeated characters with character+count. (b) Decompresses the encoded string back to the original. (c) If the compressed string is not shorter, returns the original.

Constraints:

- Single characters without count still output as single char.
- Handle empty strings.
- Decompressor should handle all valid RLE strings.

Skills Assessed:

- String traversal and comparison
- Count accumulation logic
- String building and integer conversion

Sample Input :

aabbbccddddee

Expected Output

Compressed: a2b3c2d4e2

Decompressed: aabbbccddddee

For input 'abc': Compression not beneficial.

Original:

Your Code

```
s = input()

if s == '':
    print("Compressed:")
    print('Decompressed:')
else:
    compressed = ''
    count = 1

    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            compressed += s[i - 1]
            if count > 1:
                compressed += str(count)
            count = 1

    compressed += s[-1]
    if count > 1:
        compressed += str(count)

    if len(compressed) < len(s):
        print('Compressed:', compressed)

        decompressed = ''
        i = 0
        while i < len(compressed):
            ch = compressed[i]
            i += 1

            num = ''
            while i < len(compressed) and compressed[i].isdigit():
                num += compressed[i]
                i += 1

            if num == '':
                decompressed += ch
            else:
                decompressed += ch * int(num)

        print('Decompressed:', decompressed)
    else:
        print('Compression not beneficial.')
        print('Original:', s)
```

Input

```
aabbbccddddee
```

Output

```
Compressed: a2b3c2d4e2
Decompressed: aabbbccddddee
```

```python

text = input()

stop = ['the', 'is', 'a', 'an', 'in', 'of', 'and', 'to', 'for']

paragraphs = len([p for p in text.split('\n') if p.strip() != ''])
sentences = len(re.findall(r'[.!?]', text))
if sentences == 0:
    sentences = 1

words = re.findall(r'\b[a-zA-Z]+\b', text)
total_words = len(words)

total_length = 0
freq = {}
syllables = 0

for w in words:
    word = w.lower()
    total_length += len(word)

    if word not in stop:
        if word in freq:
            freq[word] += 1
        else:
            freq[word] = 1

    count = 0
    prev = False
    for ch in word:
        if ch in 'aeiou':
            if not prev:
                count += 1
            prev = True
        else:
            prev = False
    if count == 0:
        count = 1
    syllables += count

avg_word = round(total_length / total_words, 2) if total_words else 0
avg_sentence = round(total_words / sentences, 2) if sentences else 0

score = round(206.835 - 1.015 * (total_words / sentences) - 84.6 * (syl

print('Words:', total_words)
print('Sentences:', sentences)
print('Paragraphs:', paragraphs)
print('Avg Word Length:', avg_word)
print('Avg Sentence Length:", avg_sentence)

top = sorted(freq.items(), key=lambda x: (-x[1], x[0]))[:5]
print('Top 5 Words:')
for w, c in top:
    print(w, c)

print('Flesch Reading Ease:", score)
```

Input

```
Python is a powerful programming language
```

Output

```
Words: 6
Sentences: 1
Paragraphs: 1
Avg Word Length: 6.0
Avg Sentence Length: 6.0
Top 5 Words:
```

Submitted